

NPE Incident Triage — E2E Solution Architecture

1. Objective & Scope

Automatically triage NPE incidents reported in the **Triage-NPE** Teams channel (or filed as Jira/ServiceNow tickets) — without a human chasing the owning team or root cause first. The service **diagnoses, never claims a fix**: it ingests the incident, classifies it against a source-system catalog (deterministic rules first, LLM as fallback), correlates it against other recent incidents on the same application within a rolling 24-hour window, executes the approved diagnostic playbook for that (source_system, category), synthesizes a root-cause assessment with recommended actions — LLM/RAG-driven and grounded on operator feedback, not a hardcoded rule tree, once activated — and reports back to the channel, the ticket, and the owning team, with an operator UI for supplying context, giving feedback, and retrying.

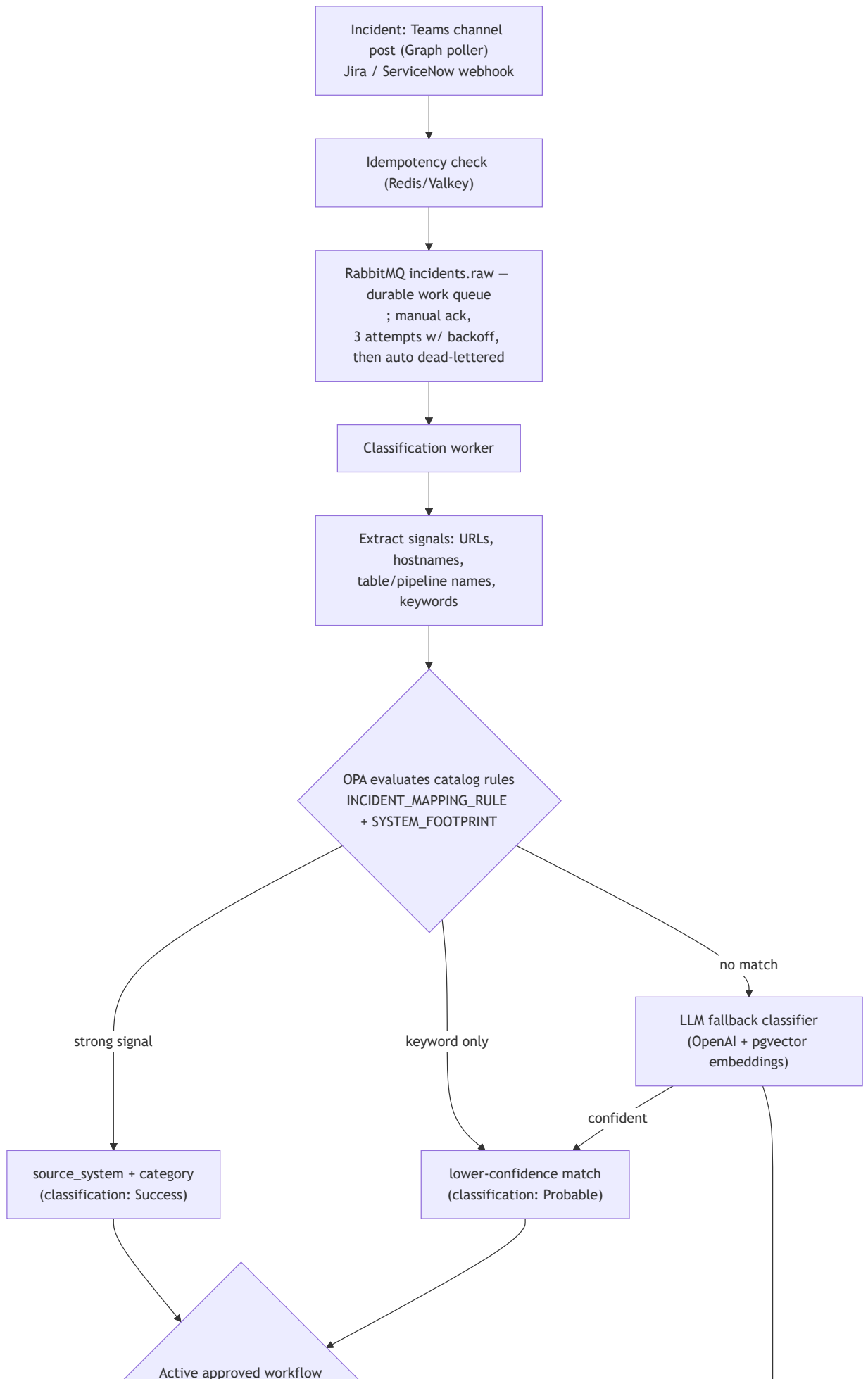
target scale : 3,000 incidents / 24h across 300 applications, each deployed in 6 non-prod environments. This number is load-bearing, not aspirational — it's why correlation's window is a full rolling day per application (not a short burst window), why RCA synthesis needs to decouple from the request path.

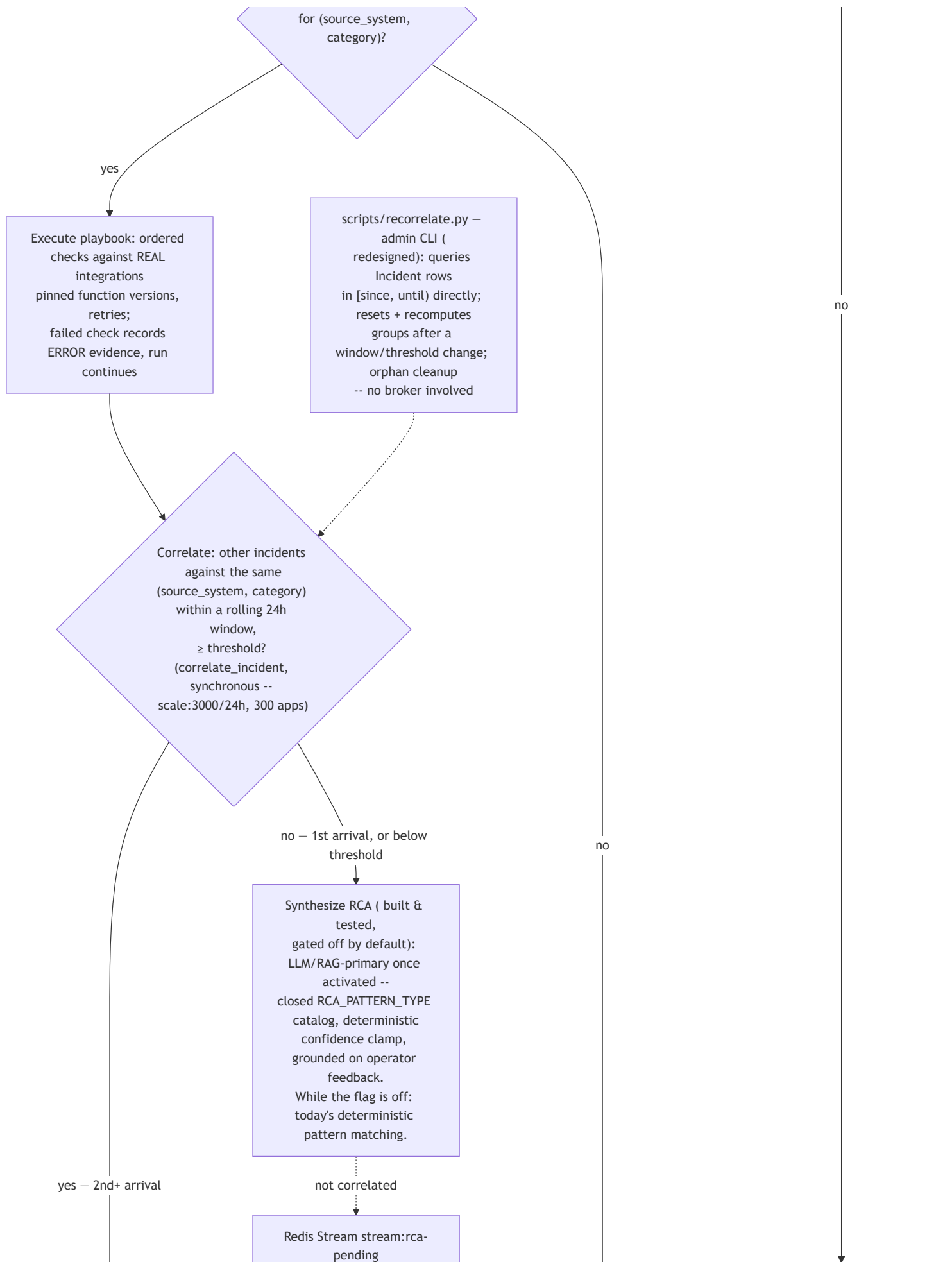
2. What Users Can Achieve (Use Cases / User Stories)

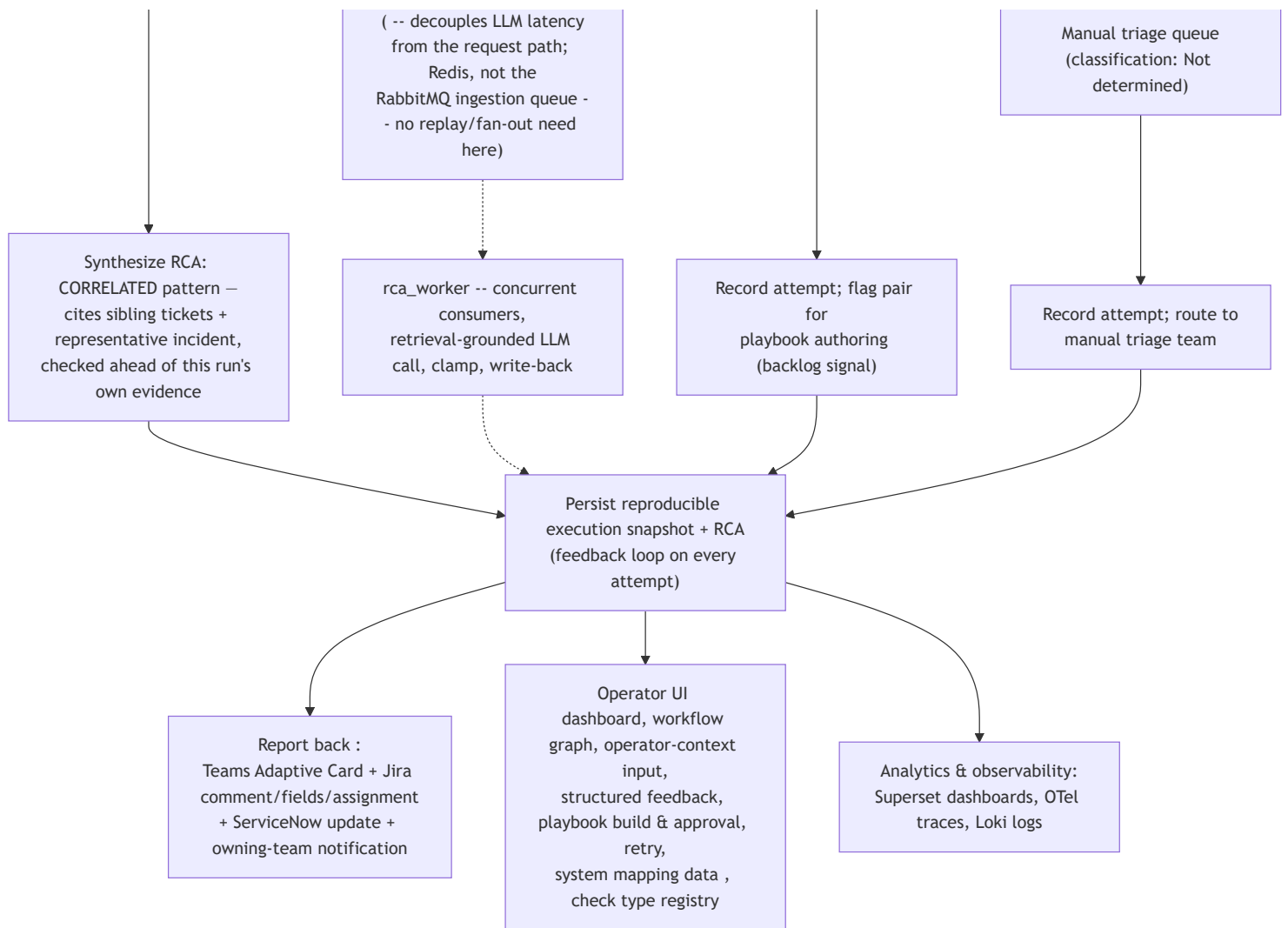
Who	As a... I can...	So that...
QA / UAT reporter	post an incident (standard template) in DSG-Triage-NPE or file a Jira ticket, and receive the RCA back in the channel/ticket — suspected system, category, root cause, contributing factors, recommended actions	I don't chase multiple teams to find who owns the problem or what caused it; triage time drops from hours to minutes
QA / UAT reporter	see an honest outcome every time — even "inconclusive, manual investigation needed" with the diagnostic evidence attached	I always know the next step; nothing disappears into a black box
QA / UAT reporter	when several people hit the same application within the same day, get back "correlated with N other incidents against this system today" citing the sibling tickets, instead of N independently-worded (and possibly conflicting) verdicts	I immediately recognize it's one shared outage, not my own isolated problem
Triage operator	view a dashboard of all triaged incidents with the latest RCA, filters, and live stats (total / success / failed / unidentified / avg process time)	I see the triage backlog and quality at a glance, and spot trends
Triage operator	open any incident's workflow run: a pictorial, clickable graph of every check with its inputs and outputs (evidence)	I can follow exactly how the conclusion was reached and trust or challenge it
Triage operator	supply additional context (a hint, a known related change) when triggering or retrying an RCA, and see it	I can steer a diagnosis with something only a human would know, without

Who	As a... I can...	So that...
	factored into the synthesis alongside the automated evidence	editing the evidence myself
Triage operator	give feedback on any RCA — a comment, a confidence score 1–5, and (once RCA synthesis is LLM-driven) an optional structured correction to the diagnosed pattern or status	the team continuously improves rules and playbooks based on real outcomes, and my corrections actually change future RCA output, not just sit in a table
Triage operator	re-run an incident's RCA, optionally choosing a different (approved) workflow version or corrected mapping, and see the full retry history per incident	mis-classified or inconclusive incidents are cheap to re-triage as playbooks improve — no ticket re-filing
Playbook author / admin	browse the function registry — all 20 diagnostic checks with their parameters and code logic — then compose a new playbook by picking checks in order	creating or improving a diagnostic playbook needs no engineering deploy; it is submitted, rendered for review, and only runs once approved
Owning engineering squad	receive the RCA, recommended actions, and assignment on our ticket / channel notification	we start fixing instead of re-diagnosing; the addressed team is routed automatically
Platform / management	extend coverage by inserting catalog rows (new source system, footprints, mapping rules, playbook) — and replay the incident backlog after rule improvements	the solution scales to all NPE systems without code changes, and every executed run is a reproducible, auditable snapshot
Platform / management	tune how aggressively incidents are correlated (time window, minimum cluster size) and replay history through the new setting via an admin CLI, without touching classification or re-running any playbook	correlation quality improves the same way rule quality does — as a config/data change, not a deploy, and past incidents benefit retroactively

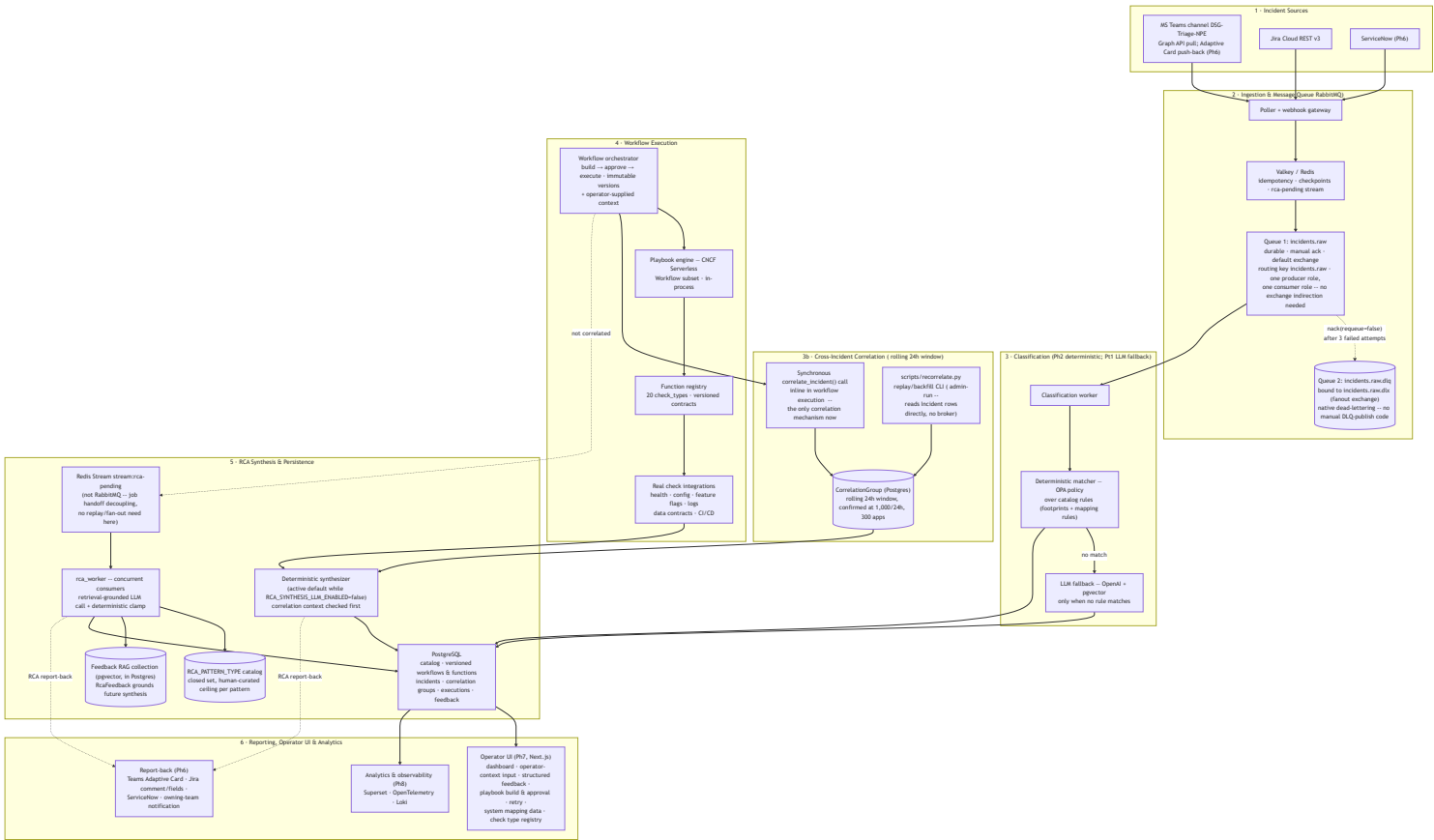
3. Functional Flow (Target)





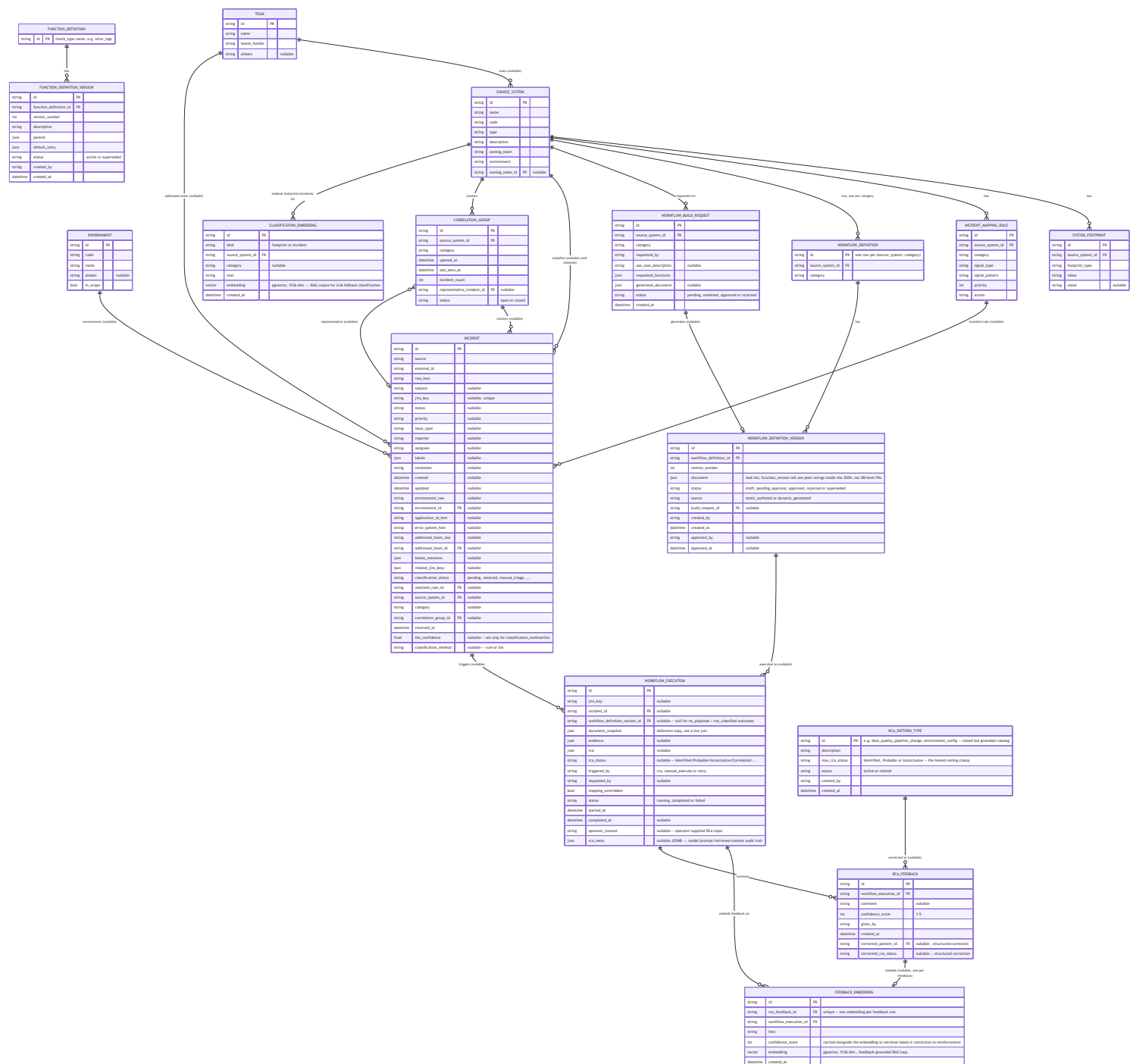


4. Solution Architecture (Target)



6. Software Matrix — What Runs What

What each piece of software in the stack is actually responsible for, and its build status. This is the reference to check before assuming a diagram box names the right technology



Layer / Functionality	Software / Technology	Role in this system	Status
Language & runtime	Python $\geq$ 3.12, managed by <code>uv</code>	Everything runs through <code>uv sync / uv run</code> ; no separate venv/pip workflow	Built
API framework	FastAPI	Webhook endpoints, full REST control plane (catalog, functions, workflows, incidents, stats, health)	Built
Data access	SQLAlchemy (async <code>AsyncSession</code>)	ORM over Postgres for every table; <code>Base.metadata.create_all</code> today, no migrations tool yet	Built
Primary datastore	PostgreSQL 17 — <code>pgvector/pgvector:pg17 image</code>	System of record: catalog, incidents, correlation groups, workflow/function versions, executions, feedback	Built
Vector similarity search	pgvector (Postgres extension)	RAG embeddings for LLM classification fallback (<code>ClassificationEmbedding</code>) and RCA feedback grounding (<code>FeedbackEmbedding</code>),	Built (schema + embed-on- write); LLM read-path gated off by default
Cache / ephemeral state	Valkey (Redis-compatible), <code>redis.asyncio client</code>	Ingestion idempotency de-dup; <code>stream:rca-pending job</code> handoff decoupling RCA- synthesis latency from the request path	Built
Message queue	RabbitMQ , <code>aio-pika client</code>	<code>incidents.raw</code> durable work queue + <code>incidents.raw.dlq</code> dead-letter queue.	Built
Policy engine	Open Policy Agent (OPA), <code>opa/policies/mapping.rego</code>	Deterministic classification rule evaluation against catalog data (<code>INCIDENT_MAPPING_RULE</code> + <code>SYSTEM_FOOTPRINT</code>)	Built
LLM / RAG	OpenAI SDK (<code>AsyncOpenAI</code>)	Classification fallback and RCA synthesis (Part 2); structured output via <code>chat.completions.parse</code> ; embeddings via <code>embeddings.create</code>	gated off by default (<code>LLM_FALLBACK_ENABLED</code> , <code>RCA_SYNTHESIS_LLM_ENABLED</code>) pending real check integrations

Layer / Functionality	Software / Technology	Role in this system	Status
Workflow orchestration	In-process CNCF Serverless Workflow subset (<code>app/playbook_engine.py</code>)	Executes approved playbooks — ordered checks, retries, continue-on-error.	Built
Diagnostic checks	20 async <code>check_type</code> implementations, one file per check under <code>app/check_types/</code> (e.g. <code>pipeline_status_v2.py</code>), dispatched via <code>app/check_implementations.py</code>	Per-integration diagnostics — health, config/feature-flag, logs, data contracts, CI/CD, connectivity, etc.	Stubbed — every evidence line marked <code>[STUBBED]</code> ;
Operator UI	Next.js (React), server-side <code>/api/backend/*</code> rewrite	Dashboard, incident/execution detail, workflow graph, playbook list + builder (<code>/playbooks/build</code>), operator-context input, feedback, retry, System Mapping Data (<code>/system-mapping</code> , Check Type Registry (<code>/check-types</code>)	Built
Teams ingestion	Microsoft Graph API, MSAL app-only auth	Polls DSG-Triage-NPE channel messages for incident posts	Built
Jira ingestion	Jira Cloud REST v3	Webhook + poller Jira-key extraction and field sync; opt-in RCA comment posting	Built
ServiceNow ingestion	ServiceNow REST API	Ticket ingestion/update	Planned
Local dev orchestration	Docker Compose	Brings up postgres, valkey, rabbitmq, opa, frontend as one stack (<code>docker compose up -d</code>)	Built
Backend test infrastructure	pytest + <code>testcontainers</code>	Full integration suite against real Postgres/Valkey/RabbitMQ/OPA containers, session-scoped fixtures — not mocks	Built
Frontend E2E testing	Playwright (<code>frontend/playwright.config.ts</code> , <code>frontend/e2e/</code>)	Covers build→approve, dashboard→detail navigation, retry-with-mapping-override, System Mapping Data CRUD incl. the blocked-delete (409)	Built — 5 specs, all passing against the live stack

Layer / Functionality	Software / Technology	Role in this system	Status
		path, and Check Type Registry version-switching	
CI pipeline	— none configured —	Automated test/build execution on push/PR	Not built — no .github/workflows/ or equivalent anywhere in the repo; tests run manually today
Observability	OpenTelemetry, Grafana Loki, Apache Superset	Traces, logs, analytics dashboards over incident/execution data	Planned — zero code footprint
Schema migrations	Alembic	Versioned schema migrations, replacing today's Base.metadata.create_all	Planned — no alembic.ini /migrations directory exists
API authentication	— none configured —	Auth/authorization on the REST control plane itself	Planned — API is trusted- network-only today; MSAL elsewhere in the stack only covers the Teams Graph poller, a separate trust boundary